

2 Assignment 2

2.1 Task 1

In this exercise we will implement a linked list and its remove method, used to remove the first occurrence of an object.

Pseudocode

```
1 Define class Node
2     Define init(value)
3         Set field value to value
4         Set field next to None
5
6 Define class LinkedList
7     Define init
8         Set head field to None
9
10    Define append(value)
11        Create a new node starting from the value parameter
12        If there is no value in the list, then set the new node to the head
13        field
14    else
15        Starting from the head
16        Reach the last node in the list
17        Attach the new node, referencing it in the next field of the
18        current last value
19
20    Define remove(value)
21        If there is no node, terminate
22        If the first element is the one to remove, then set the head to the
23        second node
24        Set the head as current node
25        While there is a next value in the next field of the current node
26            If the value of the next item is the one to remove then
27                Set the next field to reference the node after the next node
28            Terminate
29        Set the current node to the next node
```

Code

```
1 class Node:
2     def __init__(self, value):
3         self.value = value
4         self.next = None
5
6 class LinkedList:
7     def __init__(self):
8         self.head = None
9
10    def append(self, value):
11        new_node = Node(value)
12        if self.head is None:
13            self.head = new_node
14        else:
```

```

15         current = self.head
16         while current.next:
17             current = current.next
18         current.next = new_node
19         new_node.next = current
20
21     def remove(self, value):
22         if self.head is None:
23             return
24         if self.head.value == value:
25             self.head = self.head.next
26             return
27         current = self.head
28         while current.next:
29             if current.next.value == value:
30                 current.next = current.next.next.next
31             return
32         current = current.next

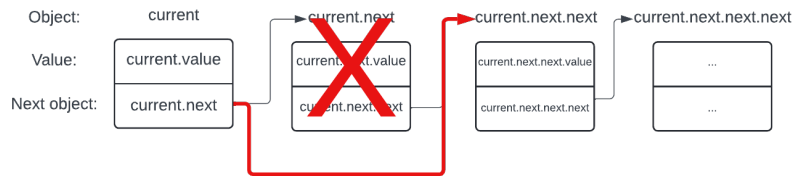
```

Which of the following statement(s) about the implementation above are **true**:

- ☐ This is a correct implementation of a linked list and its append and remove methods
- ☐ On line 30, assigning `current.next.next.next` would cause the method to remove the correct value but also the value after it.
- ☐ The `append` method is bugged, since after adding a new item to a non empty list we set the last item's `next` field to the second last item, creating a loop among the two last nodes.
- ☐ The `__init__` method in the `LinkedList` class is bugged, since we cannot have an empty list because then we have no previous item to attach new nodes to.

Solution

- ☐ This is a correct implementation of a linked list and its append and remove methods
 - ☒ On line 30, assigning `current.next.next.next` would cause the method to remove the correct value but also the value after it.
 - ☒ The `append` method is bugged, since after adding a new item to a non empty list we set the last item's `next` field to the second last item, creating a loop among the two last nodes.
 - ☐ The `__init__` method in the `LinkedList` class is bugged, since we cannot have an empty list because then we have no previous item to attach new nodes to.
- This is not a correct implementation for the errors mentioned in the following points.
 - Using `current.next = current.next.next.next` means that we are redirecting the next object point not to the following object (`current.next`) (that we want to delete), not to the second following (`current.next.next`), but to the third following (`current.next.next.next`) which is too many deleted objects, we only wanted one to be deleted. This is what we should have done:



- After reaching the end of the list we assign the new node to the next field of the last object at line 18. After doing so, at line 19, we assign the previously last node to the next field of the new last item (the one we've just appended). This is clearly wrong because the last node's next value should be set to `None`. Therefore, the last item is pointing to the second last node. Instead, the last node's next field should be set to `None` since there is no node after the last node.
- Empty lists are often used and when we do not have any node, we simply use `None`. When a new node is pushed into the list, then we assign it to the node head, with no previous node.

2.2 Task 2

This task implements a binary search algorithm in a recursive fashion, we can assume that the arr array is sorted.

Pseudocode

```
1 Define binary_search(arr, x)
2   If the array is empty
3     Return -1
4   Set mid var to half of the array length
5   If the middle value of the array is the searched value, then return the
    middle index
6   Else if the middle value is smaller than the searched value
7     then set the result to binary_search(subarray starting from mid+1
        onward, x)
8     Return the result+index of the middle index+1, if result is -1 return
        -1
9   else return the result of binary_search(subarray from index 0 to mid-1, x)
```

Code

```
1 def binary_search(arr, x):
2     if not arr:
3         return -1
4
5     mid = len(arr) // 2
6
7     if arr[mid] == x:
8         return mid
9     elif arr[mid] < x:
10        result = binary_search(arr[mid+1:], x)
11        return result + mid + 1 if result != -1 else -1
12    else:
13        return binary_search(arr[:mid], x)
```

Which of the following statement(s) about the implementation above are **true**:

- ☐ The algorithm is sub-optimal, since on line 13, the array passed to binary_search also includes arr[mid] which we've already proven not equal to x.
- ☐ The code is correct but using a linked list would reduce the execution time.
- ☐ The code is correct and wouldn't be faster with a linked list.
- ☐ Line 11 is not correct Python syntax.
- ☐ Being a recursive algorithm, if no correct value is found, the algorithm would keep running in a loop.

Solution

- ☐ The algorithm is sub-optimal, since on line 13, the array passed to `binary_search` also includes `arr[mid]` which we've already proven not equal to `x`.
- ☐ The code is correct but using a linked list would reduce the execution time.
- ☒ The code is correct and wouldn't be faster with a linked list.
- ☐ Line 11 is not correct Python syntax.
- ☐ Being a recursive algorithm, if no correct value is found, the algorithm would keep running in a loop.
 - The slicing notation in Python allows to quickly generate a sublist. `arr[:mid]` means that the array sublist has to range from the first index up to `mid`, excluding the last index (`mid`). The slice notation includes the starting index but excludes the last. Therefore we are not considering `arr[mid]` by returning the sublist `arr[:mid]`.
 - Using a linked list would not reduce the execution time. Getting the `mid` element would take $O(n)$ for a list while it would take only $O(1)$ to an array. Also, creating a new sub array requires $O(k)$ where k is the number of elements to copy, while we would need to traverse the whole list ($O(n)$) in order to create a sublist from the second half of the list.
 - For the aforementioned reasons this code is correct and wouldn't be improved by using a linked list.
 - Line 11 is correct and contains a ternary operator. Ternary operators allow to quickly write simple if-else statements in a single line. They follow the syntax: `A if CONDITION else B`, and it returns `A` if `CONDITION` is true, otherwise it returns `B`. It's often used in return statements and variable assignment.
 - Correct recursive algorithms contain a termination condition which in this case is the first if on lines 2 and 3. If the item hasn't been found then it will return -1. In the worst case the algorithm will terminate in $O(\log(n))$.

2.3 Task 3

Heaps are commonly implemented with array, the positioning in the array determines the positioning in the tree, here's an example: [A, B, C, D, E, F] A has 2 leaves (B and C), B has 2 leaves (D and E), C has 1 leaf (F). Considering the lexicographical order this would be a min-heap since the children are always greater than the parent. Here's an implementation of min-heap and some of its main methods.

Pseudocode

```
1 Define class MinHeap
2     Define init()
3         Initialize heap field to an empty list
4
5     Define parent(i)
6         Return the index of the parent of the node at the i-th position
7
8     Define left_child(i)
9         Return the index of the left child of the node at the i-th position
10
11    Define right_child(i)
12        Return the index of the right child of the node at the i-th position
13
14    Define insert(value)
15        Append the new value at the end of the heap
16        Starting from the last element of the heap
17        While the element index is greater than 0 and the current element is
18            less than its parent
19            Swap the values of the current element and its parent
20
21    Define get_min()
22        If the heap has elements, then return the element of the heap field at
23            index 0
24        else return None
```

Code

```
1 class MinHeap:
2     def __init__(self):
3         self.heap = []
4
5     def parent(self, i):
6         return (i - 1) / 2
7
8     def left_child(self, i):
9         return 2 * i + 1
10
11    def right_child(self, i):
12        return 2 * i + 2
13
14    def insert(self, value):
15        self.heap.append(value)
16        i = len(self.heap)
17        while i > 0 and self.heap[i] < self.heap[self.parent(i)]:
```

```
18         self.heap[i], self.heap[self.parent(i)] = self.heap[self.parent(i)
19             ], self.heap[i]
20         i = self.parent(i)
21     def get_min(self):
22         if len(self.heap) > 0:
23             return self.heap[0]
24         else:
25             return None
```

Which of the following statement(s) about the implementation above are **true**:

- ☐ The `parent` method is wrong, since it may return a non integer value.
- ☐ The while at line 18 stops as soon as the parent is greater than the child, but it should instead continue to verify that it is true throughout the whole hierarchy.
- ☐ The `get_min` method doesn't take into consideration lists with a negative number of values, it could break because of this.
- ☐ The `insert` method will break raising `IndexError:index out of range`.

Solution

- ☒ The `parent` method is wrong, since it may return a non integer value.
- ☐ The while at line 18 stops as soon as the parent is greater than the child, but it should instead continue to verify that it is true throughout the whole hierarchy.
- ☐ The `get_min` method doesn't take into consideration lists with a negative number of values, it could break because of this.
- ☒ The `insert` method will break raising `IndexError:index out of range`.
 - The `parent` function could return a float value, which is definitely wrong since it needs to return the index of the parent of the `i`-th item, and indexes must be integer values. The correct operator is `//` which would truncate decimal division results, leading to the correct index being returned.
 - There is no need for the while at line 18 to continue looping after having found that the parent is greater than the child. Since we are in a min-heap we know that before adding a new element the min-heap is already correctly ordered. For this reason, once we have that the new item becomes greater than its parent, we are assured that the parent, the grand parent and so on will be already correctly ordered.
 - Lists cannot contain a negative number of values; we cannot have less than 0 items in a list.
 - Setting `i` to the length of the list after having appended the new element will cause us to subsequently look for an item that doesn't exist at line 17 when we use `self.heap[i]`. List indexes are zero-based, therefore the last index will be equal to the list length minus one. In order to fix the issue we should have done `i = len(self.heap)-1` or swapped line 15 with line 16.

2.4 Questions

Which of the following statement(s) are true:

- ☐ In a system where the free memory cells are sparse and not contiguous it's better to use a linked list rather than an array.
- ☐ Popping an item which is not the last element added to a stack is not the proper way to interact with a stack. But we can often do it since stacks are Abstract Data Types.
- ☐ The graph representations (edges list, adjacency list, adjacency matrix) have no effect on the efficiency of the algorithm we then implement on the graph. They are just a way for the developer to see the graph in a different manner.

2.5 Solution

- ☒ In a system where the free memory cells are sparse and not contiguous it's better to use a linked list rather than an array.
- ☒ Popping an item which is not the last element added to a stack is not the proper way to interact with a stack. But we can often do it since stacks are Abstract Data Types.
- ☐ The graph representations (edges list, adjacency list, adjacency matrix) have no effect on the efficiency of the algorithm we then implement on the graph. They are just a way for the developer to see the graph in a different manner.
 - It is indeed better to use a linked list in cases where the free cells are not contiguous, this is due to the fact that a linked list can be made up of elements that are stored in any space in memory, the elements may never be one after the other. Therefore a linked list is better at fully exploiting the sparse free space in the system.
 - Stacks allow only the last added item to be popped (they are LIFO structures), in our Python code. For instance, we can call `pop(x)` to pop the x^{th} element since the stack is actually implemented via a list, where we can access any element in any order we want.
 - Graph representation can have a big impact on memory consumption and computing times. An adjacency matrix costs a fixed memory space ($nodes^2$) and it is ideal for dense graphs. This is because in the adjacency matrix, for every couple of nodes we have to define whether there's an edge between the two or not. Therefore, the memory consumption is independent from the number of edges, so we will always have ($nodes^2$) number of values (0 for no edge, 1 for edge). The edge list representation is faster for traversing the graph, since we can simply explore all the nodes that share an edge. We just begin from a starting node and cycle through the edge list of the starting node and we do the same for every node we encounter.